Simon's Algorithm

Finding a hidden secret, explained simply — with Qiskit code

QUANTUM v1.0 2026

Simon's algorithm was the first quantum algorithm to show a truly **exponential** speedup over any classical method. It is not the most famous quantum algorithm, but it is the most important one historically: it directly inspired Shor's factoring algorithm, which is the reason people worry about quantum computers breaking encryption. This guide explains it in plain language, walks through a worked example, and gives you code you can run.

1 The Problem, in Everyday Terms

Imagine a very strange vending machine. You punch in a code, and it hands you a snack. The machine has a quirk: **every snack comes out for exactly two different codes**, never one, never three.

You want to discover the machine's secret rule. It turns out the two codes that give the same snack are always related the same way — by a fixed hidden pattern we will call `s` . If you know one code, you can find its partner by applying `s` to it.

Your goal: figure out the secret pattern s, using as few button presses as possible.

What ‘applying s’ means

Codes are binary strings, and ‘applying s’ means **XOR** — comparing bit by bit and writing 1 where they differ, 0 where they match. It is like a light switch: XOR with 1 flips the bit, XOR with 0 leaves it alone.

```
code 001 XOR s 110 = partner 111
```

So the machine's rule is:

```
f(x) = f(x XOR s) for every code x
```

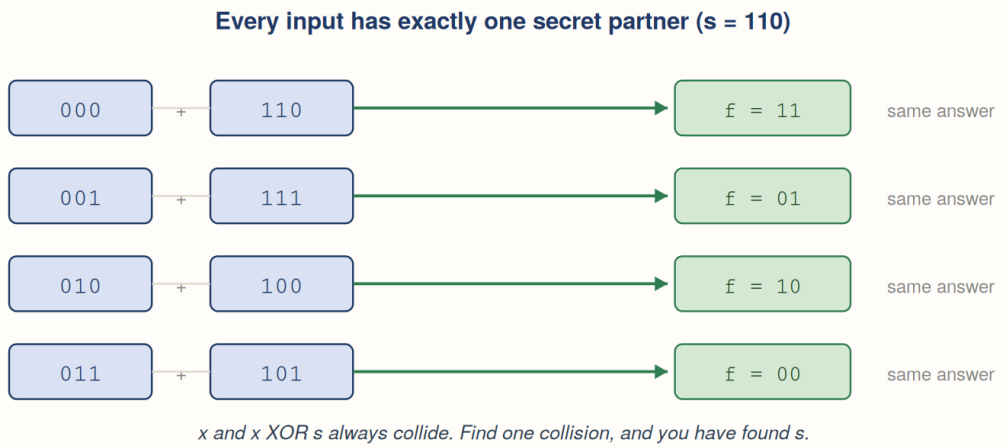

Figure 1. With s = 110, every code is paired with exactly one partner, and the pair shares an answer.

■ The everyday comparison

Think of a room full of people where everyone has exactly one twin, and twins always wear matching shirts. You cannot see who is related to whom. You want to work out the *rule* that decides who twins with whom. Finding a single matching pair of shirts tells you a lot.

2 Why This Is Hard for a Normal Computer

A normal computer has only one strategy: press buttons and watch for a repeat. Try a code, note the snack. Try another, note the snack. Keep going until two codes give the same snack. Once you find such a pair, XOR them together and you have `s` immediately.

The trouble is the waiting. With a code that is n bits long, there are 2 to the power n possible codes. You are hunting for a collision among a huge number of possibilities. On average you must press buttons roughly 2 to the power n-over-2 times — an **exponential** number of tries. Add one bit to the code and the work multiplies.

● The birthday comparison

This is the classic ‘birthday problem’. In a room of 23 people there is a coin-flip chance two share a birthday, even though there are 365 days. Collisions show up sooner than you would guess — but ‘sooner than you would guess’ is still exponential when the number of possibilities is 2 to the power n.

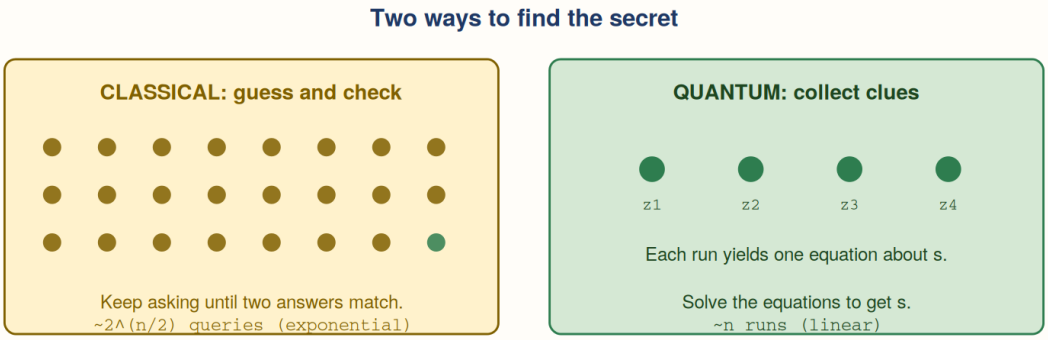

Figure 2. The classical approach hunts blindly for a collision. The quantum approach gathers clues instead.

Approach	How many times you use the machine	Growth
Classical (guess & check)	About 2 to the power n-over-2	Exponential — doubles with every extra bit.
Quantum (Simon's algorithm)	About n	Linear — one more bit, one more run.

> ■ **Put concretely**
> For a 100-bit secret, the classical method needs roughly a million-billion tries. Simon's algorithm needs about 100 runs. That gap is what ‘exponential speedup’ means.

3 How the Quantum Version Works

Here is the crucial thing to understand, and it surprises most people: **the quantum computer never tells you the secret**. It gives you a clue. You run it several times, collect several clues, and then finish the job with ordinary pencil-and-paper algebra.

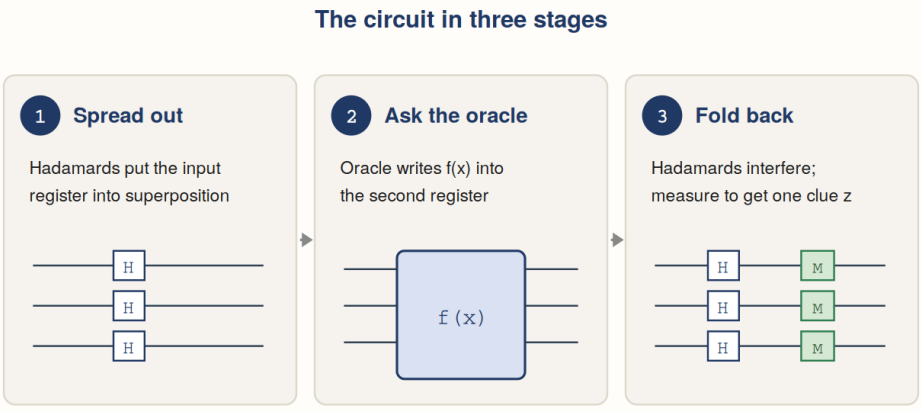

Figure 3. The circuit has three stages, repeated about n times.

Stage 1 — Spread out

Apply a Hadamard gate to each qubit in the first register. This puts them into **superposition**, meaning the register now represents every possible code at once rather than any single one.

Stage 2 — Ask the machine

Feed the superposition through the oracle. It writes the corresponding snack into a second register. Because the input was every code at once, the second register now holds every snack at once — and crucially, the two codes that share a snack become **linked** to each other.

Stage 3 — Fold back and look

Apply Hadamard gates to the first register again. This causes **interference**: the paired codes overlap, and their overlap cancels out every possibility that disagrees with the secret. When you measure, you get a bitstring `z` — and it is guaranteed to satisfy one specific equation.

$$z \cdot s = 0 \pmod 2$$

In words: line up z and s, multiply matching bits, add the results, and the total is always even. That is your clue. It does not name s, but it rules out half of the possibilities.

> **■ The single most common misunderstanding**
>
> Superposition does **not** mean the computer ‘tries every answer and picks the right one’. If you measured a plain superposition you would get a random result and learn nothing. The real work is done by **interference** in stage 3, which arranges for all the wrong answers to cancel each other out before you look.

● The noise-cancelling comparison

Interference is what noise-cancelling headphones do. Two sound waves meet; where a peak lines up with a trough, they cancel to silence. Simon's algorithm arranges the maths so the wrong answers are peaks meeting troughs, and only the useful clues survive.

4 Turning Clues into the Answer

Each run gives one clue. Collect roughly n of them (a few extra runs cover the occasional useless all-zeros result), and you have a system of simultaneous equations. Solving it is ordinary school algebra, just done with even/odd arithmetic instead of the usual kind.

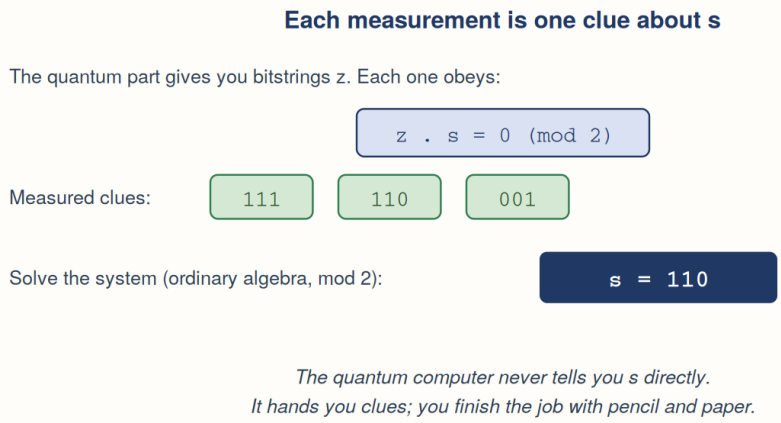

Figure 4. The quantum computer supplies clues; a normal computer solves them.

This division of labour — quantum sampling followed by classical solving — is a pattern you will see again in Shor's algorithm. It is the template.

5 Worked Example: s = 110

Our secret is three bits long, so we need three input qubits and three output qubits: six in total. Every code pairs with its XOR-110 partner.

Code x	Partner x XOR 110	They share an answer
000	110	yes
001	111	yes
010	100	yes
011	101	yes

Running the circuit 1024 times produced only four distinct measurements. That is not a coincidence — exactly half of all eight possible bitstrings are ruled out by the secret.

Measured z	Check z . s (mod 2)	Allowed?
000	0	yes (but useless — tells us nothing)
001	0	yes — a real clue
110	0	yes — a real clue
111	0	yes — a real clue
010, 011, 100, 101	1	never appear

Feed the three useful clues into the solver and out comes `s = 110` . Verified.

> ■ **Watch for the all-zeros result**
> The bitstring 000 satisfies the equation trivially and carries no information. Expect to throw away roughly one run in every few, and plan on a handful of extra runs to compensate.

6 Runnable Qiskit Code

Verified on **Qiskit 2.5.0** with **qiskit-aer 0.17.2**. The program has both halves: the quantum circuit that gathers clues, and the classical solver that turns them into the answer.

Installation

```
windows / python 3.13
C:\python3_13\python.exe -m pip install qiskit qiskit-aer
```

Part 1 — The quantum circuit

simon.py

```python
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

def simon_oracle(s):
    # Builds the 'vending machine' for secret s.
    n = len(s)
    qc = QuantumCircuit(2 * n)
    s_rev = s[::-1]      # Qiskit counts qubits right-to-left

    # copy the input register onto the output register
    for i in range(n):
        qc.cx(i, n + i)

    # XOR the secret in, so x and x^s collide
    if '1' in s_rev:
        j = s_rev.index('1')      # pick a control qubit
        for i in range(n):
            if s_rev[i] == '1':
                qc.cx(j, n + i)
    return qc


def simon_circuit(s):
    n = len(s)
    qc = QuantumCircuit(2 * n, n)

    # Stage 1: spread out
    for i in range(n):
        qc.h(i)
    qc.barrier()

    # Stage 2: ask the machine
    qc.compose(simon_oracle(s), inplace=True)
    qc.barrier()

    # Stage 3: fold back, then look
    for i in range(n):
        qc.h(i)
    qc.measure(range(n), range(n))
    return qc
```

Part 2 — The classical solver

This is Gaussian elimination — the same row-reduction you would do by hand — but every addition is XOR, because we are working with even/odd arithmetic.

simon.py (continued)

```python
def solve_for_s(equations, n):
    # Row-reduce the clues over even/odd arithmetic.
    rows = [[int(b) for b in z] for z in equations if '1' in z]
    pivots = []
    r = 0
    for c in range(n):
        piv = None
        for i in range(r, len(rows)):
            if rows[i][c] == 1:
                piv = i
                break
        if piv is None:
            continue
        rows[r], rows[piv] = rows[piv], rows[r]
        for i in range(len(rows)):
            if i != r and rows[i][c] == 1:
                rows[i] = [a ^ b for a, b in zip(rows[i], rows[r])]
        pivots.append(c)
        r += 1

    free = [c for c in range(n) if c not in pivots]
    if len(free) != 1:
        return None          # need more clues
    f = free[0]
    s = [0] * n
    s[f] = 1
    for i, c in enumerate(pivots):
        s[c] = rows[i][f]
    return ''.join(map(str, s))
```

Part 3 — Run it

```
simon.py (continued)
secret = '110'          # change to any binary string
n = len(secret)

qc = simon_circuit(secret)
print(qc.draw(output='text'))

sim = AerSimulator()
counts = sim.run(transpile(qc, sim), shots=1024).result().get_counts()
print('Counts:', counts)

# drop the useless all-zeros clue
equations = [z for z in counts if '1' in z]
print('Clues:', equations)

recovered = solve_for_s(equations, n)
print('Recovered s:', recovered)
print('Actual    s:', secret)
print('Match:', recovered == secret)
```

Verified output

```
stdout
Counts: {'000': 254, '111': 271, '110': 255, '001': 244}
Clues: ['111', '110', '001']
Recovered s: 110
Actual    s: 110
Match: True
```

Notice the counts land on only four of the eight possible bitstrings, each with roughly a quarter of the shots. The four forbidden strings never appear at all. That is the interference doing its work.

■ The endianness trap

The `s[::-1]` reversal matters. Qiskit numbers qubit 0 as the *rightmost* character of a bitstring. Omit the reversal and the oracle encodes a mirrored secret, so the measured results will violate the z-dot-s equation. This is the bug that catches almost everybody, including on the first attempt at this very document.

● Tip — how to check your own implementation

For every measured z, compute the dot product with your known secret s and confirm it is even. If any measurement gives an odd result, your oracle is wrong. This test catches endianness bugs immediately.

7 Why Simon's Algorithm Matters

Reason	Explanation
The first exponential separation	Earlier algorithms (Deutsch-Jozsa, Bernstein-Vazirani) showed quantum computers could be faster. Simon's was the first to show they could be *exponentially* faster than any classical method, including randomized ones.
It inspired Shor's algorithm	Peter Shor read Simon's result and recognized the same structure in factoring. Shor's algorithm — the one that threatens RSA encryption — is a direct descendant.
It established the template	Quantum sampling followed by classical post-processing. Almost every major quantum algorithm since follows this shape.
It teaches the hidden subgroup problem	Simon's secret string is a simple case of a broad mathematical pattern that also covers factoring and discrete logarithms.
Benchmarking	Like Bernstein-Vazirani, the answer is known in advance, so it makes a clean test of whether real quantum hardware is behaving.

> ■ **The honest caveat**
> Nobody has a practical vending machine to interrogate. Simon's problem is artificial — constructed to demonstrate a point. Its value is that the point it demonstrates turned out to be enormous.

8 Simon's vs Bernstein-Vazirani

The two are cousins, and it is worth seeing them side by side.

	Bernstein-Vazirani	Simon's
Registers	n qubits + 1 ancilla	n qubits + n qubits
Oracle output	One bit	An n-bit string
Runs needed	Exactly 1	About n
Classical post-processing	None — read the answer directly	Solve a system of equations
Classical cost of the problem	n queries (linear)	2 to the power n-over-2 (exponential)
Speedup	Modest (n → 1)	Exponential

The key structural difference: Bernstein-Vazirani's oracle answers with a single bit, so one clever query extracts everything. Simon's oracle answers with a whole string, hiding the secret in a *relationship between inputs* rather than in the output itself. That is why you need several runs and some algebra.

9 Glossary

Term	Plain-language definition
XOR	Compare two bits: write 1 if they differ, 0 if they match. Like a light switch — XOR with 1 flips, XOR with 0 leaves alone.
Oracle	The black box you are allowed to query. Here, the vending machine that turns a code into a snack.
Two-to-one function	A rule where exactly two inputs always give the same output. Never one, never three.
Superposition	A qubit holding several possibilities at once. Created by Hadamard gates.
Hadamard gate (H)	The gate that creates superposition. Applied twice, it undoes itself — which is why it appears at both ends.
Interference	Possibilities cancelling each other out, like noise-cancelling headphones. This is where the real work happens.
Measurement	Looking at a qubit, which collapses it to one definite answer. You only get to look once per run.
mod 2	Even/odd arithmetic. 1+1 = 0, because two is even.
Gaussian elimination	The standard method for solving simultaneous equations by systematically cancelling variables.
Exponential speedup	The classical cost doubles with each extra bit; the quantum cost grows by a fixed step.
Little-endian	Qiskit's convention: qubit 0 is the rightmost character of a measured bitstring. The source of most bugs.

10 References

- Simon, D. R. — *On the Power of Quantum Computation*, SIAM Journal on Computing (1997).
- Shor, P. W. — *Algorithms for Quantum Computation: Discrete Logarithms and Factoring* (1994).

- Qiskit SDK documentation — docs.quantum.ibm.com
- Qiskit Aer simulator — qiskit.github.io/qiskit-aer
